Saturday 5:53 PM

> Olá, preciso de 3 artigos recentes que usam algoritmos genéticos para geração música midi. Eu pretendo usar o piano roll como representação dos indivíduos, então pode dar preferência pra esse tipo de representação, mas pode ser outra se não tiver com esse.

Sim. Fiz uma busca focada em **2024–2026**, procurando especificamente trabalhos que usem **Genetic Algorithms (GA) / Evolutionary Algorithms para geração ou recombinação de música simbólica/MIDI**. Há uma observação importante: **é relativamente raro encontrar trabalhos recentes em que o indivíduo inteiro seja literalmente uma matriz piano roll**. Muitos usam uma representação simbólica baseada em notas/eventos e depois mostram o resultado em piano roll.

Os **3 que considero mais relevantes para o seu projeto** são:

1. Evolutionary Music Monophony Generator Using Non-Binary Representation Genes from Genetic Algorithm — 2024

Soong et al., 2024

Artigo — SAGE

É provavelmente o mais interessante para você do ponto de vista de **representação genética**.

O trabalho propõe representar cada nota musical como um **gene não-binário**, incorporando informações como:

- nota/pitch;
- duração;
- acidente;
- sequência temporal.

O GA então evolui sequências desses genes para produzir melodias monofônicas. Os autores destacam justamente que uma representação não-binária permite representar informação musical de maneira mais direta do que codificar tudo em bits. Sage Journals

Representação:

```
Plain text
Individual = [gene1, gene2, gene3, ..., geneN]
```

```
gene = informação musical
       ├── pitch
       ├── duração
       └── acidente
```

Não é piano roll, mas é muito útil para você porque mostra uma alternativa de **cromossomo estruturado musicalmente**.

---

2. M⁶(GPT)³: Generating Multitrack Modifiable Multi-Minute MIDI Music from Text using Genetic Algorithms, Probabilistic Methods and GPT Models — 2024/2025

Poćwiardowski, Modrzejewski & Tatara

Artigo no arXiv

Esse é particularmente relevante porque trabalha **diretamente no domínio MIDI** e gera músicas **multitrack e de vários minutos**.

O sistema utiliza GPT para transformar uma descrição textual em parâmetros musicais e depois emprega, entre outras técnicas, um **genetic algorithm para geração dos elementos melódicos**. A representação da composição inclui elementos como:

- escala;
- progressão de acordes;
- melodia;
- baixo;
- acompanhamento;
- motivo;
- percussão;
- estrutura temporal.

O GA usa mutações musicalmente significativas e uma função de fitness baseada em características musicais desejadas. arXiv

O trabalho posteriormente apareceu no **IEEE ICME Workshops 2025**. ResearchGate

Implementação/demo no GitHub

Para seu trabalho: é interessante principalmente como referência de **GA + MIDI + geração de composição longa**.

3. Advancing Music Recombination with Enhanced Genetic Algorithm: A Focus on Aesthetics and Computational Efficiency — 2025

Sanjay Majumder, ICMC 2025

Artigo/PDF

Este é provavelmente o **mais próximo da ideia que você está buscando**.

O trabalho recebe **dois arquivos MIDI** e utiliza suas características para evoluir novas composições. São extraídas características como:

- melody;
- harmony;
- rhythm;
- duration;
- chromatic interval;
- velocity;
- tonalidade;
- etc.

Essas características são utilizadas na função de fitness, enquanto seleção, crossover e mutação produzem novas músicas. ResearchGate

E há um detalhe muito interessante para você:

> O artigo apresenta explicitamente os resultados gerados **em piano roll**.

Os autores mostram, por exemplo, piano rolls de músicas geradas a partir de diferentes pesos entre *Amore Senza Fine* e *Black Coffee*. ResearchGate

Além disso, o artigo é de **2025** e foi publicado nos Proceedings of the **50th International Computer Music Conference (ICMC)**. ResearchGate

---

Comparando os três

Artigo	Ano	MIDI	GA	Piano roll	Multitrack
Evolutionary Music	2024	✓	✓	—	—

Artigo	Ano	MIDI	GA	Piano roll	Multitrack
Monophony Generator					
M⁶(GPT)³	2024/25	✓	✓	—	✓
Advancing Music Recombination	2025	✓	✓	✓*	✓

* O piano roll é utilizado para visualizar/analisar os resultados; a representação interna do GA é mais baseada em características/eventos MIDI do que em uma matriz piano roll pura.

E tem um quarto que eu acho especialmente importante

Durante a busca encontrei também o trabalho que originou o terceiro:

Music Recombination using a Genetic Algorithm, de **Sanjay Majumder e Benjamin D. Smith**.

Ele é muito interessante para a sua ideia porque os indivíduos são efetivamente construídos como **sequências de notas MIDI**. O artigo explica que são inicialmente criados **500 indivíduos**, cada um contendo uma sequência de notas MIDI, e a mutação substitui notas utilizando um conjunto de notas extraídas dos arquivos MIDI de entrada. ResearchGate

E novamente há resultados apresentados em **piano roll**. ResearchGate

O trabalho mais recente de 2025 é justamente uma evolução desse método.

Sobre sua ideia de usar Piano Roll

Eu acho que **piano roll é uma representação muito boa para o seu caso**, principalmente porque você quer fazer algo como:

> **GA → indivíduo → música MIDI → fitness → evolução**

Você poderia definir:

$$M \in \mathbb{R}^{T \times P}$$

onde:

- T = número de passos temporais;
- P = número de pitches;
- $M[t, p]$ = velocity da nota p no instante t.

Por exemplo:

```
                Pitch
               C5 C#5 D5 D#5 E5 F5 ...
time 0          0   0  80   0  0  0
time 1          0   0   0   0  90  0
time 2          0   0   0   0  90  0
time 3          0   0  0   0   0  0
...
```

Isso tem uma vantagem enorme para o seu projeto: **crossover e mutação podem ser feitos diretamente sobre regiões musicais**.

Por exemplo:

```
Pai A:
[████████████] [            ] [████████████]

Pai B:
[            ] [████████████] [            ]

               ↓ crossover

Filho:
[████████████] [████████████] [████████████]
      A              B              A
```

Você poderia inclusive fazer **crossover por compassos**, em vez de crossover aleatório por célula.

E isso fica particularmente interessante se sua intenção continuar sendo **imitar um artista específico**: o fitness poderia medir simultaneamente características de **melodia, ritmo, harmonia, densidade,**

intervalo, duração e estilo.

Minha recomendação

Para começar seu experimento, eu usaria:

```
Individual
    ↓
Piano Roll
    ↓
MIDI
    ↓
Feature Extraction
    ↓
Fitness
    ├── Similaridade melódica
    ├── Similaridade rítmica
    ├── Harmonia
    ├── Densidade
    ├── Intervalos
    └── Características do artista
    ↓
Selection
    ↓
Crossover
    ↓
Mutation
```

Dos artigos acima, eu começaria pelo Majumder 2025, porque ele está muito próximo da ideia de **recombinar características de músicas MIDI usando GA**, e pelo Soong et al. 2024 para estudar **como estruturar o cromossomo**. O $M^6(GPT)^3$ seria uma boa referência para a parte de **geração MIDI multitrack e composição mais longa**.

> Ótimo, vou olhar os artigos. Mas algumas observações que eu pretendo fazer: Cada indivíduo deve ser capaz de representar tanto notas simples quanto acordes e múltiplas pistas de instrumentos, não só uma única pista. Por exemplo: 4 pistas - piano, string, baixo, bateria.

Sim — e essa observação muda bastante a forma como eu estruturaria o indivíduo. Para o seu caso, eu **não usaria um único piano roll 2D**. Eu usaria um **piano roll multitrack**, em que cada indivíduo contém uma matriz para cada instrumento.

Por exemplo, para 4 pistas:

```
Indivíduo
│
├── Piano
│   └── Piano Roll [tempo × pitch]
│
├── Strings
│   └── Piano Roll [tempo × pitch]
│
├── Bass
│   └── Piano Roll [tempo × pitch]
│
└── Drums
    └── Drum Roll [tempo × instrumento]
```

Formalmente:

$$I = \{M_{piano}, M_{strings}, M_{bass}, M_{drums}\}$$

onde:

$$M_k \in \mathbb{R}^{T \times P_k}$$

Notas simples e acordes

Essa representação resolve naturalmente a questão de **nota vs. acorde**.

Uma nota:

```
Tempo →
       0  1  2  3
C4     0  90  0  0
```

Um acorde de C maior:

```
Tempo →
       0  1  2  3
C5     90  0  0  0
G4     90  0  0  0
E4     90  0  0  0
C4     90  0  0  0
```

Não precisamos definir explicitamente `"acorde = C major"` . O acorde **emerge da simultaneidade das notas**.

Isso é uma vantagem importante para um algoritmo genético, porque mutações podem atuar diretamente sobre as notas.

E múltiplas pistas

Você poderia ter algo assim:

Plain text

```
                Piano     Strings     Bass      Drums
                _____     _______     ____      _____
Compasso 1        ♫          —         ♩         •
Compasso 2        ♫          ♫         ♩        • •
Compasso 3        ♩          ♫         ♩         •
Compasso 4        ♫          —         ♩        • •
```

Então o cromossomo seria essencialmente:

$$\boxed{I = \begin{bmatrix} M_{piano} \\ M_{strings} \\ M_{bass} \\ M_{drums} \end{bmatrix}}$$

Mas há uma diferença importante: **bateria não deveria usar exatamente a mesma dimensão de pitch das outras pistas**.

Para piano, strings e baixo, você pode ter:

$$M \in \mathbb{R}^{T \times 128}$$

usando os 128 valores MIDI.

Para bateria, seria melhor:

$$M_{drums} \in \mathbb{R}^{T \times D}$$

onde D representa os instrumentos de percussão MIDI utilizados, por exemplo:

```
Plain text

36 Kick
38 Snare
42 Closed Hi-Hat
46 Open Hi-Hat
49 Crash
51 Ride
...
```

Eu faria uma pequena mudança na definição do indivíduo

Em vez de pensar:

> **Indivíduo = piano roll**

eu pensaria:

> **Indivíduo = uma composição MIDI completa representada por um conjunto de piano rolls.**

Isso é muito mais adequado ao que você está querendo fazer.

```
Plain text

Individual
│
├── Metadata
│   ├── BPM
│   ├── Time signature
│   ├── Key
│   └── ...
│
└── Tracks
    ├── Piano
    │   └── Piano Roll
    ├── Strings
    │   └── Piano Roll
    ├── Bass
    │   └── Piano Roll
```

```
      └── Drums
          └── Drum Roll
```

E isso abre uma possibilidade **muito interessante para o crossover**.

Em vez de fazer crossover célula por célula, você pode fazer:

Crossover temporal

```
Plain text

Pai A                          Pai B

| comp. 1 | comp. 2 | comp. 3 | comp. 4 |
     ↓           ↓
     └─────────────┘
          crossover

Filho

| comp. 1 | comp. 2 | comp. 3 | comp. 4 |
|    A    |    A    |    B    |    B    |
```

aplicado **simultaneamente às quatro pistas**.

Assim você preserva relações musicais:

```
Plain text

Piano     ──────── A ────────|──── B ────────
Strings   ──────── A ────────|──── B ────────
Bass      ──────── A ────────|──── B ────────
Drums     ──────── A ────────|──── B ────────
                             ↑
                         crossover
```

Isso é melhor do que misturar aleatoriamente as células das quatro matrizes, porque você mantém **frases musicais inteiras**.

E para o seu objetivo de **imitar um artista específico**, eu acho que essa representação é particularmente promissora: o fitness pode avaliar a música inteira e também cada pista individualmente.

Por exemplo:

$$F = w_1 F_{melody} + w_2 F_{rhythm} + w_3 F_{harmony} + w_4 F_{instrumentation} + w_5 F_{style}$$

Assim, você não está simplesmente tentando gerar uma sequência de notas; está evoluindo uma **composição multitrack MIDI completa**.

> Essa abordagem pode exigir muito recurso computacional? Serão matrizes bem grandes.

Sim, **pode ficar grande**, mas dá para estruturar de uma forma bastante eficiente. Na verdade, eu não me preocuparia tanto com a quantidade de pistas; o principal problema será a **resolução temporal × duração da música × tamanho da população**.

Por exemplo, suponha:

- 4 instrumentos;
- 128 pitches;
- 1 passo = 1/16 de semínima;
- 120 BPM;
- música de 2 minutos.

Teríamos aproximadamente:

$$120/4 \times 2 \times 60 = 3600$$

passos temporais.

Uma representação ingênua seria:

$$4 \times 3600 \times 128 = 1.843.200$$

células por indivíduo.

Se usar `float32`:

$$1.843.200 \times 4 \approx 7,4\ MB$$

por indivíduo.

Com uma população de 100 indivíduos:

$$\approx 740\ MB$$

só para armazenar os indivíduos.

E isso ainda não inclui cópias temporárias durante crossover/mutação, fitness, MIDI etc.

Mas eu não usaria `float32`

Para MIDI, isso seria desperdício.

Se cada célula representa apenas:

```
Plain text
0   = silêncio
1-127 = velocity
```

você pode usar:

```
Python
np.uint8
```

Então aquele mesmo indivíduo ocuparia aproximadamente:

$$1.843.200 \times 1 \approx 1,8\ MB$$

e uma população de 100:

$$\approx 180\ MB$$

Isso já fica bem mais tranquilo.

Mas existe uma solução ainda melhor

Eu consideraria **não representar todos os 128 pitches**.

Por exemplo, para:

```
Plain text
```

```
Piano
Strings
Bass
```

você pode definir ranges específicos:

```
Plain text
```

```
Piano   : MIDI 21–108  → 88 pitches
Strings : MIDI 36–96   → 61 pitches
Bass    : MIDI 28–60   → 33 pitches
```

E bateria:

```
Plain text
```

```
Drums   : ~10–20 instrumentos
```

Assim:

$$M_{piano} \in \mathbb{R}^{T \times 88}$$

$$M_{strings} \in \mathbb{R}^{T \times 61}$$

$$M_{bass} \in \mathbb{R}^{T \times 33}$$

$$M_{drums} \in \mathbb{R}^{T \times D}$$

Isso reduz bastante a memória.

Porém, existe um problema mais importante

O **piano roll denso** é extremamente redundante.

Imagine:

```
Plain text
```

```
C4  | 0 0 0 0 80 80 80 80 80 80 0 0 ...
E4  | 0 0 0 0 80 80 80 80 80 80 0 0 ...
G4  | 0 0 0 0 80 80 80 80 80 80 0 0 ...
```

Você está armazenando:

```
Plain text

80 80 80 80 80
```

repetidamente para representar a duração de uma única nota.

Mas MIDI originalmente pode representar isso simplesmente como:

```
Plain text

NoteOn  C4  velocity=80
duration = 5 steps
```

Então há uma troca:

Representação	Memória	Operações GA	Facilidade
Piano roll denso	🔴 alta	🟢 muito fácil	🟢
Piano roll binário	🟡 média	🟢 muito fácil	🟢
Piano roll `uint8`	🟡 média	🟢 fácil	🟢
Lista de notas MIDI	🟢 baixa	🟡 mais difícil	🟡
Eventos MIDI	🟢 muito baixa	🔴 mais difícil	🔴

Eu faria um híbrido

Para o seu projeto, eu acho que uma solução muito boa seria:

Representação genética:

```
Individual
│
├── Piano
│   └── matriz uint8
│
├── Strings
│   └── matriz uint8
│
├── Bass
│   └── matriz uint8
│
└── Drums
    └── matriz uint8
```

mas com uma resolução temporal **musical**, não necessariamente frame-a-frame.

Por exemplo:

```
1 passo = 1/16 de compasso
```

ou:

```
1 passo = 1/16 de semínima
```

Então você consegue representar:

- notas;
- acordes;
- duração;
- velocity;
- polifonia;
- múltiplos instrumentos.

E depois:

```
Piano rolls
     ↓
   MIDI
     ↓
   WAV
     ↓
avaliação
     ↓
fitness
```

E isso combina muito bem com algoritmo genético

Você nem precisa manter a população inteira como matrizes gigantes em memória.

Pode fazer:

```
Plain text

Population
   ↓
Fitness
   ↓
Selection
   ↓
Crossover
   ↓
Mutation
   ↓
New population
```

mantendo apenas a população atual e a nova população.

Um detalhe que eu considero especialmente importante para sua pesquisa

Se você pretende gerar **músicas de vários minutos**, eu evitaria começar com uma matriz que represente a música inteira.

Uma alternativa muito mais escalável seria:

$$\boxed{\text{Individual} = \text{Sequence of Musical Segments}}$$

Por exemplo:

```
Plain text

Indivíduo
│
├── Intro      → 4 compassos
├── Verse      → 8 compassos
├── Chorus     → 8 compassos
├── Verse      → 8 compassos
├── Chorus     → 8 compassos
└── Outro      → 4 compassos
```

Cada segmento teria seus próprios piano rolls.

Isso também torna o **crossover musicalmente mais interessante**:

```
Plain text

Pai A:
Intro → Verse A → Chorus A → Outro A

Pai B:
Intro → Verse B → Chorus B → Outro B

Filho:
Intro → Verse A → Chorus B → Outro A
```

Em vez de simplesmente cortar uma matriz enorme no meio.

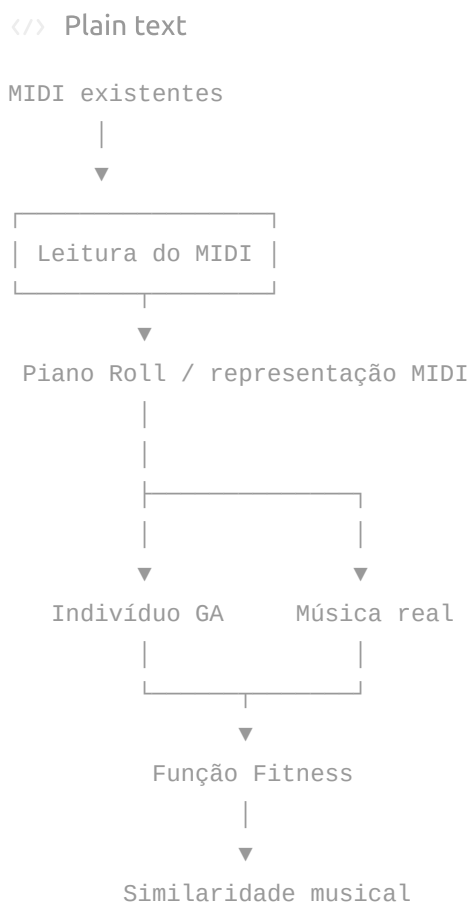
Para o seu experimento inicial, eu começaria com 4–8 compassos por indivíduo/segmento. Depois que o GA funcionar, aumentaria progressivamente a duração. Isso evita gastar GPU/CPU evoluindo músicas enormes antes de validar se a representação e o fitness realmente funcionam.

> Ao invés de usar wav, quero comparar os próprios midis gerados com midis já existentes, assim não precisa da etapa de conversão de casa indivíduo pra wav. Porém eu tenho os arquivos em .mid, que podem estar em binário, tem como eu usá-los para comparar com os indivíduos ?

Sim — e **acho que essa é uma decisão melhor para o seu algoritmo genético**. Você não precisa converter MIDI → WAV para comparar as músicas. O `.mid` pode ser lido diretamente, transformado em uma representação estruturada e comparado com o indivíduo.

E o fato de o arquivo MIDI ser "binário" **não é um problema**. MIDI é um formato binário estruturado, e existem bibliotecas Python que fazem o parsing dele.

O fluxo poderia ser



Por exemplo, usando `pretty_midi`:

```python
import pretty_midi

midi = pretty_midi.PrettyMIDI("musica.mid")

for instrument in midi.instruments:
    print(instrument.name)

    for note in instrument.notes:
        print(
            note.pitch,
            note.start,
            note.end,
```

```
            note.velocity
        )
```

Você conseguiria obter diretamente:

```
pitch
start
end
velocity
instrument
```

de cada nota.

E isso combina perfeitamente com seu piano roll
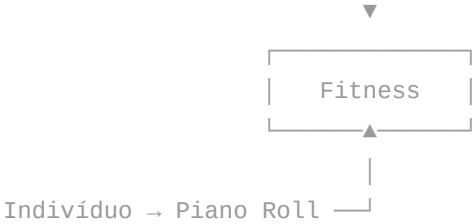
Você pode converter **uma única vez** os seus MIDI reais para a representação que será utilizada pelo GA.

Por exemplo:

```
musica01.mid
      ↓
Parser MIDI
      ↓
┌──────────┬──────────┬──────────┬──────────┐
│  Piano   │ Strings  │   Bass   │  Drums   │
└──────────┴──────────┴──────────┴──────────┘
      ↓
Piano rolls
      ↓
Salvar como .npy / .pt
```

Depois, durante a evolução:

```
MIDI real
     │
     └──→ representação pré-processada
                        │
```



Assim você **não precisa nem ficar reprocessando os MIDI reais**.

Mas há uma questão importante

Eu **não recomendaria comparar simplesmente as matrizes célula a célula**.

Por exemplo:

```
MIDI real:

C4 ──────────
E4 ──────────
G4 ──────────


Indivíduo:

C4 ──────────
E4 ──────────
G4 ──────────
```

Se houver um pequeno deslocamento temporal, uma comparação direta poderia considerar os dois muito diferentes, apesar de musicalmente serem praticamente iguais.

Por isso, eu criaria um **fitness composto**.

Por exemplo:

$$F = w_1 F_{pitch} + w_2 F_{rhythm} + w_3 F_{duration} + w_4 F_{harmony} + w_5 F_{velocity}$$

1. Pitch

Compara quais notas aparecem:

$$F_{pitch} = similarity(P_{generated}, P_{target})$$

2. Ritmo

Compara os instantes relativos das notas.

3. Duração

Compara:

```
Plain text
C4: 1/4
E4: 1/4
G4: 1/2
```

contra:

```
Plain text
C4: 1/4
E4: 1/4
G4: 1/2
```

4. Harmonia

Você pode extrair acordes ou simplesmente analisar as relações simultâneas entre pitches.

5. Velocity

Compara a dinâmica:

```
Plain text
Piano:
C4 = 60
E4 = 80
G4 = 90
```

E existe uma vantagem enorme para o seu objetivo

Você falou anteriormente que quer **gerar música imitando um artista específico**.

Nesse caso, eu não necessariamente usaria **uma música específica como target**.

Poderia ter:

Plain text



Então o GA não tenta copiar:

> "essa música exatamente"

mas tenta maximizar:

> **"o quanto esta nova composição se parece com o estilo desse artista."**

Isso é muito mais interessante cientificamente.

E você pode manter o MIDI original como fonte

Eu faria algo assim:

Plain text

```
dataset/
│
├── artist_A/
│   ├── song01.mid
│   ├── song02.mid
│   ├── song03.mid
│   └── ...
│
└── processed/
```

```
    ├── song01.pt
    ├── song02.pt
    └── song03.pt
```

Os `.mid` continuam sendo a **fonte original**, enquanto os `.pt` / `.npy` são apenas uma representação pré-processada para acelerar o GA.

Isso também permite depois reconstruir:

```
Plain text

Piano Roll
    ↓
MIDI
    ↓
DAW
```

e você consegue abrir a música gerada no MuseScore, Ableton, FL Studio, etc.

Ou seja: MIDI pode ser tanto a entrada quanto a saída do seu algoritmo, e o piano roll pode ser apenas a representação interna do indivíduo. Para o que você está planejando, considero essa arquitetura bastante adequada.

Quantas músicas midi de referência você sugere eu editar?

Se o objetivo é **treinar/evoluir um GA para imitar o estilo de um artista específico**, eu não editaria centenas de MIDI manualmente. Eu começaria com **20–30 músicas bem selecionadas**.

Uma divisão prática seria:

Quantidade	Uso
10 músicas	Protótipo inicial, verificar se a ideia funciona
20–30 músicas	⭐ Recomendo para o primeiro experimento sério
50–100 músicas	Experimento mais robusto
100+	Só depois, se a função de fitness realmente se beneficiar disso

Mas eu faria uma distinção importante

Você **não precisa necessariamente editar todas as músicas para o mesmo formato manualmente**.

Se seus MIDI já possuem:

- piano;
- strings;
- baixo;
- bateria;

você pode criar um parser que normalize automaticamente:

```
Plain text

MIDI original
      ↓
identificar instrumentos
      ↓
Piano
Strings
Bass
Drums
      ↓
normalizar BPM
normalizar tonalidade (opcional)
normalizar resolução temporal
      ↓
Piano rolls
```

A edição manual ficaria apenas para **corrigir casos problemáticos**.

Para seu primeiro experimento

Eu escolheria **20 músicas** e faria algo como:

- 20 MIDI do mesmo artista;
- 4 pistas principais;
- trechos de 8 ou 16 compassos;
- mesma resolução temporal;
- mesma faixa de pitches por instrumento;
- `velocity` preservada;
- acordes preservados como múltiplas notas simultâneas.

Depois:

15 músicas → referência para fitness

5 músicas → teste

Assim você consegue responder uma pergunta científica interessante:

> O GA consegue gerar uma composição que apresenta características do artista sem simplesmente reproduzir trechos das músicas utilizadas durante a evolução?

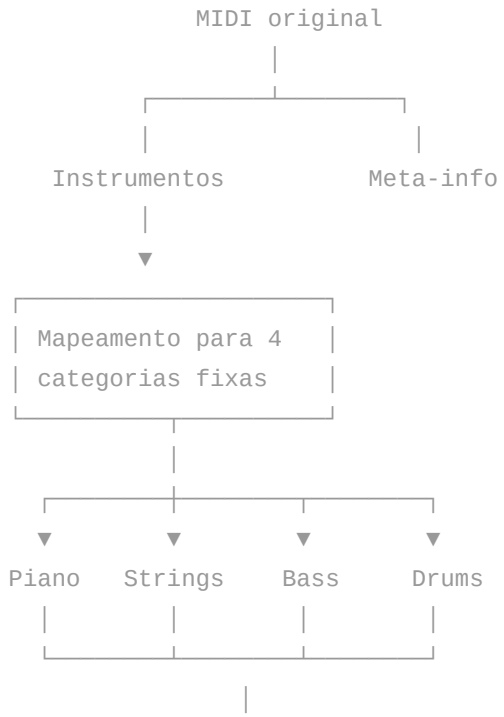
E há uma coisa que eu **não faria ainda**: editar os 20 MIDI manualmente para transformar tudo em piano roll. Primeiro eu tentaria automatizar a conversão `.mid → representação do indivíduo`. Se você conseguir isso, economiza uma quantidade enorme de trabalho e torna o experimento muito mais reproduzível.

> Porém os midi de referência tem mais do que 4 pistas e mais do que apenas 4 instrumentos. Eu preciso de alguma forma converter os midis originais para exatamente os mesmos 4 instrumentos que eu defini no meu modelo.

Exatamente. Nesse caso, eu mudaria um pouco a estratégia: **não tentaria preservar a instrumentação original do MIDI**. Você quer transformar qualquer MIDI de referência em uma representação canônica de **4 pistas**, independentemente de quantas pistas ele tenha originalmente.

Eu faria:

Plain text



▼

Piano, pelle

O ponto importante: não mapear apenas pelo nome

Por exemplo, um MIDI pode ter:

```
Plain text

Track 1: Grand Piano
Track 2: Electric Piano
Track 3: Acoustic Guitar
Track 4: Distortion Guitar
Track 5: Violin
Track 6: Cello
Track 7: Synth Pad
Track 8: Bass
Track 9: Drums
```

Você poderia definir uma regra:

Instrumentos originais	Pista do seu modelo
Piano, Electric Piano, Grand Piano	**Piano**
Violin, Viola, Cello, Strings, String Ensemble	**Strings**
Bass, Electric Bass, Synth Bass	**Bass**
Kick, Snare, Hi-Hat, Cymbal etc.	**Drums**

Então:

```
Plain text

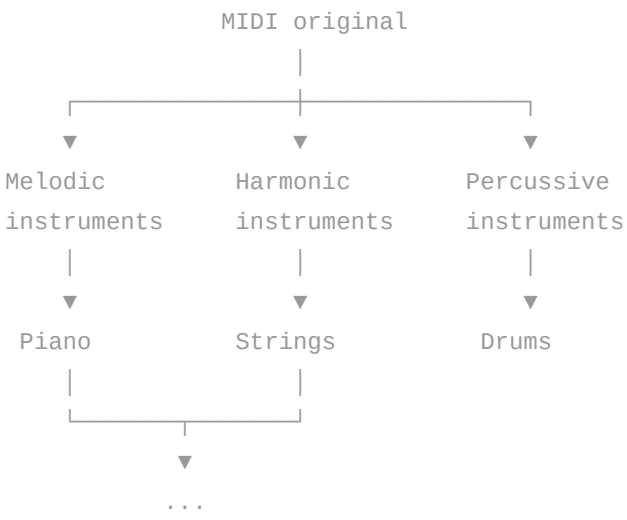
Grand Piano ──────┐
Electric Piano ───┤
                  ├──→ Piano
Violin ───────────┤
Cello ────────────┤
Synth Pad ────────┘
                  └──→ Strings
```

Mas há uma questão importante aqui: **o que fazer com guitarra, synth, saxofone, órgão etc.?**

Eu não descartaria simplesmente essas notas. Para um experimento de imitação de artista, isso poderia eliminar justamente características importantes do estilo.

Uma solução seria fazer um **mapeamento semântico**:

Plain text



Mas como você **fixou exatamente quatro instrumentos**, eu faria algo ainda mais controlado.

Uma regra que eu considero melhor

Defina **quatro papéis musicais**, não necessariamente quatro instrumentos físicos:

1. **Piano / Harmonic**
2. **Strings / Melodic**
3. **Bass**
4. **Drums**

Assim, por exemplo:

Plain text

```
Guitar
Synth
Organ
Rhodes
      ↓
  Piano/Harmonic
```

e:

Plain text

```
Violin
Viola
Cello
String Ensemble
Choir
Pad
        ↓
  Strings/Melodic
```

Isso é muito mais robusto para datasets heterogêneos.

Porém, existe um problema que precisamos resolver

Imagine que o MIDI tenha:

Plain text

```
Piano:
C4 E4 G4

Guitar:
C4 E4 G4

Strings:
C5 E5 G5
```

Se você simplesmente colocar tudo em `Piano`, teremos:

Plain text

```
Piano =
Piano + Guitar
```

e perdemos a identidade instrumental.

Mas talvez **isso não seja ruim para o seu objetivo**.

Você está tentando gerar uma composição com **quatro pistas fixas**, não necessariamente reconstruir o MIDI original instrumento por instrumento.

Então seu processo seria:

$$MIDI_{original} \rightarrow MIDI_{4tracks}$$

e **somente depois**:

$$MIDI_{4tracks} \rightarrow PianoRoll_{4tracks}$$

Eu criaria um "MIDI Normalizer"

Algo como:

```
normalize_midi.py

input.mid
   ↓
[1] Ler todas as tracks
   ↓
[2] Identificar instrumentos
   ↓
[3] Classificar instrumentos
   ↓
[4] Agrupar por papel
   ↓
[5] Resolver sobreposição
   ↓
[6] Normalizar pitch/time
   ↓
[7] Gerar exatamente 4 tracks
   ↓
normalized.mid
```

E **todos os MIDI de referência passariam por esse mesmo processo**.

Isso é importante para o experimento porque você garante:

$$\forall x \in Dataset: \quad Representation(x) = (Piano, Strings, Bass, Drums)$$

E eu faria uma coisa adicional

Não jogaria fora o MIDI original.

Manteria:

```
dataset/
│
├── original/
│   ├── music01.mid
│   ├── music02.mid
│   └── ...
│
└── normalized/
    ├── music01.mid
    ├── music02.mid
    └── ...
```

Assim você pode posteriormente comparar:

```
original MIDI
      │
      ├── análise
      │
      └── normalização
             ↓
        4-track MIDI
             ↓
          piano roll
             ↓
         algoritmo genético
```

Isso também torna seu método **reprodutível**: outra pessoa pode pegar os mesmos MIDI e aplicar o mesmo normalizador.

E uma vantagem enorme

Você pode salvar o resultado normalizado novamente como `.mid`.

Então o seu GA trabalha assim:

```
```

```
MIDI de referência
        ↓
4-track MIDI
        ↓
Piano Roll
        ↓
INDIVÍDUO
        ↓
seleção / crossover
        ↓
mutação
        ↓
novo indivíduo
        ↓
Piano Roll
        ↓
4-track MIDI
        ↓
comparação com
referências
```

Não há necessidade de WAV em nenhuma etapa.

Para o seu projeto, eu acho que essa arquitetura está ficando bem coerente: **MIDI como formato externo, piano roll multitrack como representação genética e quatro papéis instrumentais fixos como espaço de busca.**

> E se eu arriscar um indivíduo com 10 pistas, sera que fica muito pesado?

Não necessariamente. **10 pistas é perfeitamente viável**, principalmente se você usar uma representação eficiente. O problema não é tanto o número de pistas, mas **duração × resolução temporal × número de pitches × tamanho da população**.

Se você estiver pensando em um indivíduo como:

Plain text

```
Individual
├── Piano
├── Electric Piano
├── Guitar
├── Strings
├── Synth
├── Bass
```

```
├── Brass
├── Lead
├── Pad
└── Drums
```

eu diria que **10 pistas é uma escolha bastante interessante** para o seu experimento, porque preserva muito mais informação do MIDI original do que as 4 pistas.

Um cálculo aproximado

Suponha:

- 10 pistas;
- 2 minutos;
- 120 BPM;
- resolução de 16 passos por semínima;
- `uint8` ;
- 128 pitches.

Temos aproximadamente 7.680 passos temporais.

Então:

$$10 \times 7680 \times 128 = 9.830.400$$

células por indivíduo.

Como `uint8` usa 1 byte:

$$\approx 9,4\ MB$$

por indivíduo.

Uma população de 100 indivíduos:

$$\approx 940\ MB$$

Isso **já começa a ficar pesado**, mas ainda não é absurdo para uma máquina com bastante RAM.

Mas podemos reduzir muito

Você não precisa usar 128 pitches para todas as pistas.

Por exemplo:

Pista	MIDI	Pitches
Piano	21–108	88
E. Piano	21–108	88
Guitar	40–88	49
Strings	36–96	61
Synth	36–96	61
Bass	28–60	33
Brass	40–88	49
Lead	48–96	49
Pad	36–96	61
Drums	~16 peças	16

Isso reduz bastante.

E eu usaria **`uint8`**, não `float32`.

Na verdade, eu prefiro 10 pistas a 4 para seu objetivo

Pensando no que você explicou até agora, eu mudaria minha recomendação anterior.

Com apenas:

```
Plain text
Piano
Strings
```

```
Bass
Drums
```

você está fazendo uma forte compressão do estilo do artista.

Com 10 pistas você consegue capturar coisas como:

```
melodia
harmonia
baixo
bateria
sintetizadores
guitarras
pads
brass
etc.
```

Isso pode ser **muito importante para a imitação de estilo**.

A desvantagem é que o espaço de busca do GA cresce.

E há uma solução que eu acho ainda melhor

Você pode ter **10 pistas fixas, mas permitir que algumas fiquem vazias**.

Por exemplo:

```
Individual
│
├── Piano       ✓
├── E.Piano     0
├── Guitar      ✓
├── Strings     ✓
├── Synth       0
├── Bass        ✓
├── Brass       0
├── Lead        ✓
├── Pad         0
└── Drums       ✓
```

O `0` significa simplesmente:

> essa composição não utiliza essa pista.

Isso evita obrigar o GA a preencher todos os instrumentos.

E eu tomaria cuidado com uma coisa

Eu **não escolheria os 10 instrumentos arbitrariamente**.

Se você pretende trabalhar com músicas de um artista específico, primeiro faria uma análise dos MIDI dele:

```
100 MIDI
   ↓
instrumentos encontrados
   ↓
contagem
   ↓
agrupamento
```

Talvez você descubra algo como:

```
Piano          93%
Bass           91%
Drums          100%
Strings        72%
Guitar         58%
Synth          54%
Pad            47%
Lead           31%
Brass          18%
Organ          12%
```

Aí suas **10 pistas são determinadas empiricamente pelo repertório**, em vez de serem escolhidas arbitrariamente.

Isso seria inclusive uma decisão metodológica interessante para o trabalho.

Minha sugestão

Eu faria um primeiro experimento com:

10 pistas × piano roll × `uint8` × segmentos de 4–8 compassos.

Não começaria evoluindo músicas de 2–5 minutos inteiras. Primeiro validaria:

$$\text{MIDI} \rightarrow \text{10-track representation} \rightarrow GA \rightarrow \text{MIDI}$$

em pequenos segmentos.

Se funcionar, aí você aumenta a duração.

10 pistas não me parece exagero; para o seu objetivo, provavelmente é uma representação melhor que 4 pistas.

> Podemos ficar sempre na classe do instrumento como você sugeriu. Em 4 a 6 pistas. Porém como ficariam as classes? Pode ser que uma música tenha 10 pistas com varias pistas da mesma classe e outra música ter só 4, outra 6, outra 2.

Sim. E eu acho que essa é **a melhor solução**: as classes representam **funções/tipos instrumentais**, e não uma correspondência rígida com as pistas originais do MIDI.

O ponto principal é: **uma classe pode receber várias pistas do MIDI original**. Depois você agrega todas elas em uma única pista da classe.

Eu usaria 6 classes

Para um dataset musical relativamente geral:

Classe	Exemplos de instrumentos
Piano/Keys	piano, electric piano, organ, clavinet, harpsichord
Guitar	acoustic guitar, electric guitar, distorted guitar
Strings	violin, viola, cello, string ensemble
Bass	acoustic bass, electric bass, synth bass
Melodic	flute, sax, trumpet, lead synth, brass, choir

Classe	Exemplos de instrumentos
Drums	kick, snare, hi-hat, cymbals, percussion

Isso seria o **máximo de 6 pistas**.

Uma música não precisa usar todas.

O mais importante: várias pistas → uma classe

Imagine um MIDI assim:

```
Track 1   Grand Piano
Track 2   Electric Piano
Track 3   Acoustic Guitar
Track 4   Electric Guitar
Track 5   Violin
Track 6   Cello
Track 7   Bass
Track 8   Synth Bass
Track 9   Flute
Track 10  Synth Lead
Track 11  Drums
```

Depois da normalização:

```
                    Normalização
                         ↓

Piano/Keys ← Grand Piano + Electric Piano
Guitar     ← Acoustic Guitar + Electric Guitar
Strings    ← Violin + Cello
Bass       ← Bass + Synth Bass
Melodic    ← Flute + Synth Lead
Drums      ← Drums
```

Resultado:

Plain text

```
6 pistas
```

Mesmo que o MIDI original tivesse 11, 15 ou 20 pistas.

E se uma música tiver apenas 2 classes?

Perfeitamente válido.

Por exemplo:

Plain text

```
Música A

Piano/Keys  ✓
Guitar      ✓
Strings     -
Bass        ✓
Melodic     -
Drums       ✓
```

Representação:

Plain text

```
[Piano, Guitar, Bass, Drums]
```

As outras duas ficam **zeradas**.

Outra música:

Plain text

```
Música B

Piano/Keys  ✓
Guitar      -
Strings     ✓
Bass        ✓
```

```
Melodic     ✓
Drums       ✓
```

E outra:

```
Plain text

Música C

Piano/Keys  ✓
Guitar      ✓
Strings     ✓
Bass        ✓
Melodic     ✓
Drums       ✓
```

Todas continuam tendo a mesma representação:

$$6 \times T \times P$$

O que muda é o conteúdo de cada matriz.

E aqui está uma vantagem importante

Você **não precisa representar "ausência de instrumento" de maneira especial**.

Basta:

$$M_{class} = 0$$

para toda a matriz.

Por exemplo:

```
Plain text

            Piano Guitar Strings Bass Melodic Drums
Music A       ✓      ✓       0      ✓      0      ✓
Music B       ✓      0       ✓      ✓      ✓      ✓
Music C       ✓      ✓       ✓      ✓      ✓      ✓
```

Isso é muito conveniente para o algoritmo genético.

Mas existe um detalhe com a bateria

Eu manteria **Drums separada** porque bateria não é representada da mesma maneira que os instrumentos melódicos.

Enquanto:

```
Plain text
Piano:
tempo × pitch
```

a bateria seria:

```
Plain text
tempo × drum_type
```

por exemplo:

```
Plain text
Kick
Snare
Closed Hi-Hat
Open Hi-Hat
Crash
Ride
Tom
...
```

Então seu indivíduo poderia ser:

$$I = \{M_{keys}, M_{guitar}, M_{strings}, M_{bass}, M_{melodic}, M_{drums}\}$$

com:

$$M_{keys}, M_{guitar}, M_{strings}, M_{bass}, M_{melodic} \in \mathbb{R}^{T \times P}$$

e:

$$M_{drums} \in \mathbb{R}^{T \times D}$$

Eu faria uma pequena alteração no nome "Melodic"

Eu talvez chamasse essa classe de:

Other Melodic

porque ela funciona como uma categoria residual:

```
Plain text
Piano/Keys
Guitar
Strings
Bass
Other Melodic
Drums
```

Assim fica explícito que:

> instrumentos melódicos que não possuem uma classe específica vão para `Other Melodic`.

Por exemplo:

```
Plain text
Flute       → Other Melodic
Saxophone   → Other Melodic
Trumpet     → Other Melodic
Oboe        → Other Melodic
Synth Lead  → Other Melodic
Choir       → Other Melodic
```

Porém, eu faria o mapeamento baseado no General MIDI

Isso pode deixar seu pré-processamento muito mais objetivo.

O MIDI possui os **Program Numbers** do General MIDI, então podemos construir uma tabela:

```
GM Program
    ↓
Classe
```

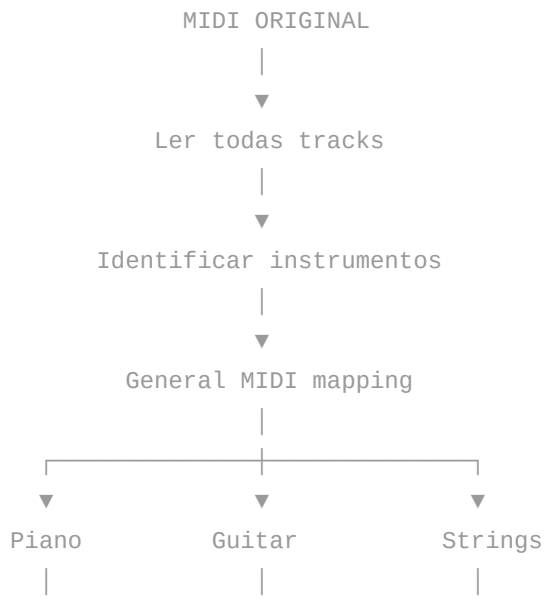
Por exemplo:

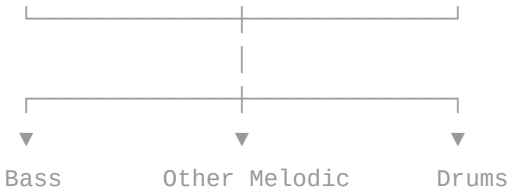
```
0–7       → Piano/Keys
24–31     → Guitar
32–39     → Bass
40–51     → Strings
52–63     → Other Melodic
...
```

E bateria é detectada pela pista MIDI de percussão/canal 10.

Isso significa que você não precisaria manualmente olhar para cada MIDI.

O pipeline que eu montaria





E aí temos uma propriedade muito boa:

$$\boxed{\text{qualquer MIDI} \rightarrow \text{mesmo espaço de representação}}$$

Se um MIDI tem 2 instrumentos, 6 ou 20, todos acabam no mesmo espaço de seis classes.

Só existe uma decisão que precisamos tomar

Quando duas ou mais pistas pertencem à **mesma classe**, precisamos decidir como agregá-las.

Por exemplo:

```
Plain text
Piano 1:
C4 = velocity 80

Piano 2:
E4 = velocity 100
```

Podemos simplesmente fazer a união:

```
Plain text
Piano:
C4 = 80
E4 = 100
```

Mas se houver conflito:

```
Plain text
Piano 1:
C4 = 70
```

```
Piano 2:
C4 = 110
```

precisamos escolher uma regra:

A. máximo

$$C4 = \max(70, 110) = 110$$

B. média

$$C4 = (70 + 110)/2 = 90$$

C. soma com clipping

$$C4 = \min(70 + 110, 127) = 127$$

Para o seu caso, eu provavelmente usaria **máximo**, porque preserva a nota com maior intensidade e evita criar artificialmente uma velocity maior pela simples existência de várias pistas.

Então eu fecharia a representação inicial assim:

```
Plain text

INDIVÍDUO
│
├── Piano/Keys
├── Guitar
├── Strings
├── Bass
├── Other Melodic
└── Drums
```

Isso te dá **6 classes fixas**, mas permite que cada música original tenha qualquer quantidade de pistas. É uma solução bem mais elegante do que tentar forçar "track 1 = piano, track 2 = strings..." e, para um GA, mantém **todos os indivíduos no mesmo espaço de busca**.

Saturday 8:20 PM

Gere um sumário final de como vai ficar todo o meu código.

Claro. Eu estruturaria o projeto como um **pipeline completo de normalização de MIDI → representação genética → algoritmo genético → MIDI gerado → avaliação**, mantendo o piano roll como representação central.

1. Estrutura geral

Plain text

```
midi_ga/
│
├── config.py
│
├── dataset/
│   ├── original/
│   │   ├── music01.mid
│   │   ├── music02.mid
│   │   └── ...
│   │
│   └── normalized/
│       ├── music01.mid
│       ├── music02.mid
│       └── ...
│
├── midi/
│   ├── reader.py
│   ├── classifier.py
│   ├── normalizer.py
│   ├── piano_roll.py
│   └── writer.py
```

---

2. MIDI original → 6 classes

A primeira etapa lê qualquer MIDI, independentemente da quantidade de tracks.

As classes seriam:

Plain text

```
1. Piano/Keys
2. Guitar
3. Strings
4. Bass
```

```
5. Other Melodic
6. Drums
```

Por exemplo:

```
Plain text
```

```
Piano
Electric Piano
Organ
      ↓
Piano/Keys

Acoustic Guitar
Electric Guitar
Distortion Guitar
      ↓
Guitar

Violin
Viola
Cello
String Ensemble
      ↓
Strings

Bass
Synth Bass
```

O mapeamento será baseado preferencialmente nos **program numbers do General MIDI**, em vez de depender do nome textual da track.

---

3. Várias tracks podem virar uma só

Isso é fundamental.

Um MIDI:

```
Plain text
```

```
20 tracks
```

pode virar:

Plain text

```
Piano/Keys      ← 3 tracks
Guitar          ← 4 tracks
Strings         ← 5 tracks
Bass            ← 2 tracks
Other Melodic   ← 3 tracks
Drums           ← 3 tracks
```

Resultado:

Plain text

```
6 tracks
```

Quando várias tracks caírem na mesma classe, suas notas serão agregadas.

Em conflitos de mesma nota/tempo, inicialmente usaríamos:

$$velocity = \max(velocity_1, velocity_2, \ldots)$$

---

4. Normalização temporal

Todos os MIDI precisam entrar no mesmo espaço temporal.

Por exemplo:

Plain text

```
1 passo = 1/16 de semínima
```

Então uma música pode ser convertida para:

Plain text

```
T × P
```

onde:

- T = passos temporais;
- P = pitches.

Isso é importante para que crossover e mutação funcionem sobre indivíduos compatíveis.

---

5. Representação do indivíduo

O indivíduo será uma **composição MIDI completa**, representada por seis matrizes.

```
Plain text
```

```
Individual
│
├── piano_keys
├── guitar
├── strings
├── bass
├── melodic
└── drums
```

Para instrumentos melódicos:

$$M_k \in \mathbb{R}^{T \times P_k}$$

Para bateria:

$$M_{drums} \in \mathbb{R}^{T \times D}$$

Inicialmente eu usaria:

```
Python
```

```
numpy.uint8
```

porque:

```
Plain text
```

```
0   → silêncio
1-127 → velocity
```

em vez de `float32`.

---

6. Notas e acordes

Não haverá uma representação especial para acordes.

Uma nota:

```
Plain text
C4
```

é simplesmente uma célula ativa.

Um acorde:

```
Plain text
C4
E4
G4
```

é representado por três notas simultâneas.

Portanto:

$$\text{acorde} = \{\text{notas simultâneas}\}$$

Isso mantém a representação simples para o GA.

---

7. Duração das notas

Aqui teremos que tomar uma decisão importante na implementação.

Uma nota longa ocupará vários passos:

```
Plain text
```

```
C4: ███████
```

em vez de apenas:

```
Plain text
C4: ▉
```

Assim o piano roll mantém:

- pitch;
- início;
- duração;
- velocity;
- polifonia.

Posteriormente podemos testar uma representação com `NoteOn/NoteOff`, mas **para a primeira versão eu manteria o piano roll explícito**.

---

8. Dataset pré-processado

Os MIDI originais serão preservados:

```
Plain text
dataset/original/
```

E criaremos versões normalizadas:

```
Plain text
dataset/normalized/
```

Por exemplo:

```
Plain text
original/song01.mid
       ↓
normalizer
```

```
        ↓
normalized/song01.mid
```

Depois podemos gerar:

```
Plain text
processed/song01.pt
```

ou:

```
Plain text
processed/song01.npy
```

contendo diretamente as seis matrizes.

Isso evita ficar fazendo parsing dos MIDI durante cada geração do GA.

---

9. Indivíduo inicial

O GA criará uma população:

```
Plain text
Population
├── Individual 1
├── Individual 2
├── Individual 3
├── ...
└── Individual N
```

Cada indivíduo terá:

```
Plain text
6 piano rolls
```

Inicialmente existem duas possibilidades:

Opção A — aleatório

Notas geradas aleatoriamente.

Opção B — baseado no dataset

Fragmentos das músicas reais são utilizados para inicializar a população.

Eu prefiro começar com B, porque um GA puramente aleatório terá um espaço de busca gigantesco.

---

10. Fitness

O fitness não será simplesmente:

```
matriz == matriz alvo
```

Vamos comparar características musicais.

Uma versão inicial:

$$F = w_p F_{pitch} + w_r F_{rhythm} + w_h F_{harmony} + w_d F_{duration} + w_v F_{velocity}$$

Por exemplo:

```
Pitch       25%
Rhythm      25%
Harmony     25%
Duration    15%
Velocity    10%
```

Os pesos serão parâmetros experimentais.

---

11. Comparação com várias músicas do artista

Em vez de comparar o indivíduo com apenas uma música:

```
Plain text

Individual
    ↓
Fitness
    ↓
Song 1
Song 2
Song 3
...
Song N
```

podemos calcular:

$$F(I) = \frac{1}{N} \sum_{i=1}^{N} F(I, S_i)$$

Assim o GA procura características **comuns ao repertório do artista**, e não copia uma música específica.

Essa é uma parte importante do experimento.

---

12. Seleção

Depois do fitness:

```
Plain text

Population
    ↓
Fitness
    ↓
Ranking
    ↓
Selection
```

Podemos começar com **tournament selection**, que é simples e funciona bem para esse tipo de problema.

---

13. Crossover

Eu evitaria crossover célula-a-célula.

Em vez disso, utilizaria **blocos temporais**.

Por exemplo:

```
Plain text

Pai A

|████|████|████|████|

Pai B

|████|████|████|████|
```

Crossover:

```
Plain text

Filho

| A  | A  | B  | B  |
```

E, principalmente, o mesmo corte temporal seria aplicado às seis pistas:

```
Plain text

Piano       A | A | B | B
Guitar      A | A | B | B
Strings     A | A | B | B
Bass        A | A | B | B
Melodic     A | A | B | B
Drums       A | A | B | B
```

Isso preserva a **coerência temporal entre instrumentos**.

---

14. Mutação

Teremos várias mutações possíveis:

Alterar pitch

Plain text

```
C4 → D4
```

Alterar velocity

Plain text

```
80 → 95
```

Adicionar nota

Plain text

```
silêncio → nota
```

Remover nota

Plain text

```
nota → silêncio
```

Alterar duração

Plain text

```
████ → █████
```

Alterar bloco

Plain text

```
compasso → outro padrão
```

Essas mutações serão probabilísticas.

---

15. Conversão do indivíduo para MIDI

Depois da evolução:

```
Individual
     ↓
6 piano rolls
     ↓
MIDI writer
     ↓
generated.mid
```

O arquivo terá seis tracks:

```
Track 1 → Piano/Keys
Track 2 → Guitar
Track 3 → Strings
Track 4 → Bass
Track 5 → Other Melodic
Track 6 → Drums
```

Assim podemos abrir diretamente o resultado em:

- MuseScore;
- DAW;
- qualquer software compatível com MIDI.

---

16. Não haverá MIDI → WAV durante o GA

Esse é um ponto importante da arquitetura.

Não teremos:

```
MIDI
 ↓
WAV
 ↓
áudio
```

```
  ↓
comparação
```

Teremos:

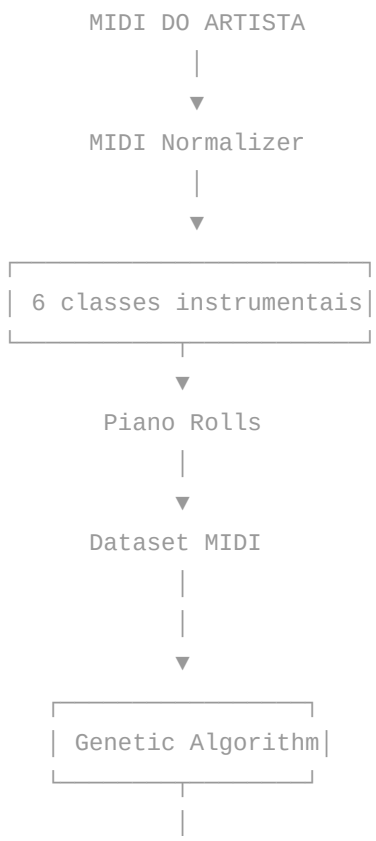
```
MIDI
 ↓
representação simbólica
 ↓
comparação
```

Isso deve tornar a avaliação **muito mais rápida**.

---

17. Fluxo completo

No final, o sistema será aproximadamente:



Configuração inicial que eu usaria

```python
NUM_CLASSES = 6

CLASSES = [
    "piano_keys",
    "guitar",
    "strings",
    "bass",
    "other_melodic",
    "drums",
]

TIME_STEP = "1/16"
VELOCITY_MIN = 1
VELOCITY_MAX = 127

POPULATION_SIZE = 100
GENERATIONS = 500

CROSSOVER_RATE = 0.8
MUTATION_RATE = 0.1
```

Mas esses números são **pontos de partida**, não valores definitivos.

A ideia central

O conceito que eu acho mais forte para o seu trabalho ficou:

$$\text{MIDI arbitrário} \rightarrow \text{6 classes instrumentais} \rightarrow \text{Piano Roll multitrack} \rightarrow \text{Indivíduo} \rightarrow \text{GA}$$

Isso resolve simultaneamente os três problemas que identificamos: **MIDI com número variável de tracks**, **múltiplos instrumentos pertencentes à mesma classe** e **indivíduos com uma representação fixa para o algoritmo genético**.

> Com relação aos dados de referência, eu uso 80% para avaliar a fitness durante a evolução, e 20% para testar separadamente após evolução?

Sim. **Essa divisão 80/20 é adequada**, mas eu faria uma pequena distinção importante: os 80% não são exatamente "dados de treino" no sentido tradicional de uma rede neural. Eles são o **conjunto de referência usado pelo fitness durante a evolução**.

Eu estruturaria assim:

```
MIDI do artista
       |
       ├──────────────┐
       |              |
      80%            20%
       |              |
       ▼              ▼
Fitness durante      Teste final
a evolução
       |              |
       ▼              |
     GA               |
       |              |
       ▼              |
Melhor indivíduo      |
       |              |
       └──────┬───────┘
              ▼
        Avaliação final
```

Os 80%

São utilizados **durante todas as gerações**:

```
População
   ↓
Fitness
   ↓
comparar com os 80%
   ↓
seleção
   ↓
crossover
   ↓
mutação
   ↓
nova população
```

Portanto, o GA "conhece" essas músicas através da função de fitness.

Os 20%

Ficam **completamente separados** durante a evolução.

Depois que o GA terminar:

```
Plain text

          GA
           |
           ▼
   melhor indivíduo
           |
           ▼
   ┌──────────────┐
   | 20% TESTE    |
   └──────────────┘
           |
           ▼
    fitness final
```

Você então pode responder:

> O indivíduo evoluído apresenta características semelhantes às músicas do artista que **não foram utilizadas durante sua evolução**?

Isso é muito mais convincente experimentalmente.

Mas eu faria uma coisa ainda melhor: 70/15/15

Se você tiver músicas suficientes, eu preferiria:

Conjunto	Uso
70%	Fitness durante evolução
15%	Validação / escolha de hiperparâmetros
15%	Teste final

O problema é que, em GA, você pode acabar fazendo várias experiências e escolhendo:

- número de gerações;

- tamanho da população;
- taxa de mutação;
- pesos do fitness;
- resolução temporal;
- número de classes;

com base nos resultados do teste.

Isso seria **data leakage**.

Com um conjunto de validação, você pode fazer:

```
Plain text
70% → desenvolver o GA
15% → escolher parâmetros
15% → resultado final
```

E o último conjunto só é usado **uma vez, no experimento final**.

Se você tiver poucas músicas

Por exemplo, se você tiver apenas **20 MIDI**:

```
Plain text
14 → evolução
3  → validação
3  → teste
```

Eu acho aceitável, mas 3 músicas é pouco para um teste estatisticamente forte.

Se tiver:

```
Plain text
50 MIDI
```

poderia fazer:

```
Plain text
35 → evolução
7  → validação
8  → teste
```

E com:

```
Plain text
100 MIDI
```

fica muito melhor:

```
Plain text
70 → evolução
15 → validação
15 → teste
```

Um detalhe importante para o seu caso

Se você dividir por **músicas**, mantenha **todos os segmentos de uma mesma música no mesmo conjunto**.

Não faça:

```
Plain text
Song A
├── segmento 1 → treino
├── segmento 2 → treino
├── segmento 3 → teste  ❌
```

Faça:

```
Plain text
Song A → treino
Song B → treino
Song C → teste
```

Porque senão o GA pode ser avaliado em um trecho de uma música que ele já "viu" durante a evolução.

Então, para o seu experimento, minha recomendação final seria 70/15/15 se você tiver uma quantidade razoável de MIDI; 80/20 é perfeitamente aceitável para o primeiro protótipo.

Como eu faria o cruzamento?

Para o seu caso, eu faria o **crossover por blocos temporais (compassos)**, e não célula por célula. Isso é importante porque você tem **6 instrumentos simultâneos** e quer preservar relações musicais entre eles.

1. Estrutura do indivíduo

Imagine que cada indivíduo tenha:

```
                TEMPO →
Piano       | A A A A | B B B B |
Guitar      | A A A A | B B B B |
Strings     | A A A A | B B B B |
Bass        | A A A A | B B B B |
Other       | A A A A | B B B B |
Drums       | A A A A | B B B B |
```

Cada coluna representa um passo do piano roll, e vários passos formam um compasso.

Suponha dois pais:

```
PAI 1:
| Compasso 1 | Compasso 2 | Compasso 3 | Compasso 4 |
|     A      |     A      |     B      |     B      |

PAI 2:
| Compasso 1 | Compasso 2 | Compasso 3 | Compasso 4 |
|     C      |     C      |     D      |     D      |
```

Um crossover possível seria:

```
FILHO:
| Compasso 1 | Compasso 2 | Compasso 3 | Compasso 4 |
```

```
|     A      |     A      |     D      |     D      |
```

Ou:

```
</> Plain text

FILHO:
| Compasso 1 | Compasso 2 | Compasso 3 | Compasso 4 |
|     C      |     C      |     B      |     B      |
```

2. O ponto mais importante

O **mesmo ponto de corte deve ser aplicado a todas as pistas**.

Por exemplo:

```
</> Plain text

             Ponto de corte
                  ↓
Piano       [===========|===========]
Guitar      [===========|===========]
Strings     [===========|===========]
Bass        [===========|===========]
Other       [===========|===========]
Drums       [===========|===========]
```

Então o filho recebe:

```
</> Plain text

             Ponto de corte
                  ↓
Piano       [ Pai 1     |    Pai 2   ]
Guitar      [ Pai 1     |    Pai 2   ]
Strings     [ Pai 1     |    Pai 2   ]
Bass        [ Pai 1     |    Pai 2   ]
Other       [ Pai 1     |    Pai 2   ]
Drums       [ Pai 1     |    Pai 2   ]
```

Isso mantém, por exemplo, **baixo + bateria + guitarra + piano sincronizados**.

---

3. Eu usaria crossover de múltiplos pontos

Para música, acho ainda melhor que um único corte.

Por exemplo, com 8 compassos:

```
Plain text
```

```
Pai 1:
A A A A | B B B B

Pai 2:
C C C C | D D D D
```

Com dois pontos de crossover:

```
Plain text
```

```
Pai 1: A A | B B | A A | B B
Pai 2: C C | D D | C C | D D

Filho:  A A | D D | A A | D D
```

Assim o GA pode combinar **frases musicais inteiras**.

---

4. Por que não fazer crossover célula por célula?

Imagine:

```
Plain text
```

```
Pai 1:
Piano:  0 0 80 0 0 90 0 0
Bass:   40 0 0 0 50 0 0 0

Pai 2:
Piano:  0 70 0 0 80 0 0 0
Bass:   0 0 40 0 0 0 50 0
```

Se você escolher aleatoriamente cada célula:

```
Plain text
```

```
Filho:
Piano:  0 70 80 0 80 90 0 0
Bass:   40 0 40 0 50 0 50 0
```

Você pode criar combinações que **não existiam musicalmente em nenhum dos pais**.

Pior ainda se o crossover for independente para cada instrumento:

```
Plain text
Piano  → corte no passo 17
Bass   → corte no passo 42
Drums  → corte no passo 31
Guitar → corte no passo 9
```

Isso destrói a estrutura temporal da música.

---

5. Eu faria uma pequena hierarquia de crossover

Para seu experimento, eu testaria **três operadores**:

Crossover 1 — Compassos

Mais conservador:

```
Plain text
| compasso | compasso | compasso | compasso |
|   P1     |    P1    |    P2    |    P2    |
```

Excelente para preservar frases.

Crossover 2 — Frases

Agrupar, por exemplo:

```
Plain text
4 compassos = uma frase
```

Então:

Plain text

```
| frase 1 | frase 2 | frase 3 | frase 4 |
|   P1    |   P2    |   P1    |   P2    |
```

Esse provavelmente seria meu **operador principal**.

Crossover 3 — Pequenos blocos

Para aumentar diversidade:

Plain text

```
| 1 comp. | 1 comp. | 1 comp. | 1 comp. |
|   P1    |   P2    |   P2    |   P1    |
```

Mas eu usaria com menor probabilidade.

---

6. Um detalhe importante: notas atravessando o corte

Existe um problema interessante.

Imagine:

Plain text

```
Pai 1:

          ┌──────────── nota
──────────┼─────────────┼──────────
          ↑             ↑
     compasso      compasso
       1             2
```

Se você cortar exatamente entre os compassos, uma nota pode começar antes do corte e terminar depois.

Eu **não cortaria a nota no meio**.

Você pode tratar cada nota como:

```
Plain text
(start, end, pitch, velocity)
```

e aplicar o crossover às notas/frases completas.

Ou, se estiver trabalhando diretamente na matriz:

- mantenha a parte da nota que pertence ao bloco;
- opcionalmente reconstrua a duração depois;
- evite criar notas artificiais muito curtas.

---

7. E os acordes?

Essa abordagem é particularmente boa para acordes.

Por exemplo:

```
Plain text
Piano

C4 ███████
E4 ███████
G4 ███████
```

O acorde inteiro está dentro de um bloco.

No crossover:

```
Plain text
Pai 1                    Pai 2
C  E  G                  D  F# A
███████                  ███████
       ↓ corte
       ________________
```

Você transfere o **bloco inteiro**, portanto o acorde permanece intacto.

---

8. Pseudocódigo

Eu implementaria algo conceitualmente assim:

Python

```
def crossover(parent1, parent2, block_size):
    """
    parent:
        [tracks, time, pitch]
    """

    _, T, _ = parent1.shape

    n_blocks = T // block_size

    child = parent1.copy()

    for block in range(n_blocks):
        start = block * block_size
        end = start + block_size

        if random.random() < 0.5:
            child[:, start:end, :] = parent2[:, start:end, :]

    return child
```

Mas eu faria uma melhoria: **não escolheria cada bloco de forma totalmente independente**. Poderia alternar segmentos:

Plain text

```
P1 P1 P2 P2 P1 P1 P2 P2
```

ou escolher pontos de crossover:

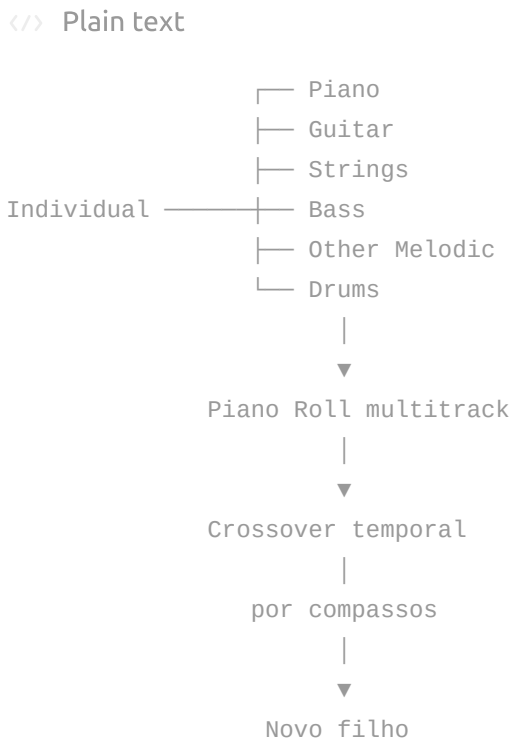
Python

```
cut1 = ...
cut2 = ...

child[:, cut1:cut2] = parent2[:, cut1:cut2]
```

Minha escolha para o seu GA

Eu começaria com:



E usaria **crossover de 2 ou 3 pontos alinhado aos compassos**, sempre aplicando os mesmos cortes às seis classes.

Isso é muito mais adequado ao seu problema do que um crossover tradicional de vetor de genes, porque o seu **gene natural não é uma célula do piano roll; é um bloco temporal musical** — idealmente um compasso ou uma frase.

Sunday 1:18 AM

Me ajude a definir um genótipo básico pra começar, com as seguintes características:

2 trilhas (piano, drum)
Duração: 30 seg

Deve permitir grupos de 1 a 5 notas (acordes)
E assuma um tempo = 120.

Sim. Para começar simples, eu evitaria representar o genótipo diretamente como uma matriz enorme. Como você quer **piano + bateria**, 30 s, acordes de até 5 notas e tempo de 120 BPM, podemos definir um genótipo baseado em **eventos musicais**.

1. Parâmetros temporais

Com:

- BPM = 120
- 4/4
- 1 batida = 0,5 s
- 1 compasso = 2 s
- 30 s = **60 batidas = 15 compassos**

Podemos usar uma resolução de **16 passos por compasso**:

```
Plain text
1 passo = 2 / 16 = 0,125 s
```

Logo:

```
Plain text
30 s = 240 passos
```

Isso dá uma resolução de semicolcheia, que é bastante conveniente para o GA.

---

2. Genótipo

Eu começaria com:

```
Plain text
Individual
│
├── Piano
│   └── 240 eventos temporais
│
└── Drums
    └── 240 eventos temporais
```

Mas cada posição do piano **não precisa ser apenas uma nota**.

Ela pode conter de **0 a 5 notas**:

```
Plain text
Piano[t] = [pitch1, pitch2, pitch3, pitch4, pitch5]
```

Por exemplo:

```
Plain text
Piano[0] = [60]
```

é C4.

Enquanto:

```
Plain text
Piano[16] = [60, 64, 67]
```

é:

```
Plain text
C4 E4 G4
```

um acorde de C maior.

E:

```
Plain text
Piano[32] = [55, 59, 62, 65, 69]
```

é um acorde de até 5 notas.

---

3. Representação que eu recomendo

Para o primeiro protótipo:

Python

```
genotype = {
    "piano": [
        [60],              # passo 0
        [],                # passo 1
        [],
        [64, 67, 71],      # passo 3
        ...
    ],

    "drums": [
        ["kick"],           # passo 0
        [],
        ["hihat"],          # passo 2
        ["snare", "hihat"], # passo 3
        ...
    ]
}
```

Ou seja, temos:

Plain text

```
              240 passos
             ──────────────────>

Piano        [ ] [ ] [C] [ ] [C,E,G] [ ] ...
Drums        [K] [ ] [H] [S,H] [ ]    [H] ...
```

Isso é muito mais fácil de manipular geneticamente do que começar diretamente com uma matriz `240 × 128`.

---

4. Genótipo do piano

Cada gene temporal poderia ser:

Python

```
PianoGene = {
    "notes": [60, 64, 67],
    "velocity": 100,
    "duration": 4
}
```

Então:

```
notes     → quais notas
velocity  → intensidade
duration  → duração em passos
```

E o piano teria até 240 posições de início:

```
Piano = [
    Gene0,
    Gene1,
    Gene2,
    ...
    Gene239
]
```

Um gene vazio seria:

```
None
```

Por exemplo:

```
piano = [
    {"notes": [60, 64, 67], "velocity": 100, "duration": 8},
    None,
    None,
    None,

    {"notes": [62, 65, 69], "velocity": 90, "duration": 8},
```

```
    ...
]
```

Isso significa:

Plain text

```
tempo
0s                                    30s
|-------------------------------------|
| C-E-G           D-F-A               |
| ███████         ███████             |
|                                     |
```

---

5. Drums

Para bateria, eu não usaria pitch arbitrário inicialmente.

Definiria um pequeno conjunto:

Python

```
DRUMS = {
    "kick": 36,
    "snare": 38,
    "hihat": 42,
    "open_hihat": 46,
    "crash": 49
}
```

Então:

Python

```
drums = [
    ["kick"],
    [],
    ["hihat"],
    ["snare", "hihat"],
    [],
    ["hihat"],
```

```
    ...
]
```

Observe que aqui pode haver **mais de um instrumento no mesmo passo**, porque kick + hi-hat simultaneamente é perfeitamente normal.

---

6. Um genótipo completo

Por exemplo:

Python

```
individual = {
    "piano": [
        {
            "notes": [60, 64, 67],
            "velocity": 100,
            "duration": 8
        },
        None,
        None,
        None,

        {
            "notes": [65, 69, 72],
            "velocity": 95,
            "duration": 8
        },
        None,
        ...
    ],

    "drums": [
```

---

7. Uma decisão importante: duração

Eu recomendo **não colocar duração como parte independente do tempo de cada célula**.

Por exemplo:

Plain text

```
passo 0
C E G
duration = 8
```

significa:

```
Plain text
```

```
C E G
■■■■■■■
```

enquanto:

```
Plain text
```

```
passo 8
D F A
duration = 4
```

significa:

```
Plain text
```

```
D F A
■■■■
```

Isso permite ao GA evoluir não apenas **quais notas aparecem**, mas também **quanto tempo elas duram**.

---

8. Tamanho do genótipo

Temos:

```
Plain text
```

```
30 segundos
120 BPM
4/4
16 passos/compasso

15 compassos
```

```
× 16 passos
= 240 posições
```

Então:

```
Plain text
```

```
Piano:
240 genes

Drums:
240 genes
```

Total:

```
Plain text
```

```
480 genes temporais
```

Mas isso **não significa apenas 480 bits**. Cada gene pode carregar informações musicais.

Piano

```
Plain text
```

```
Gene =
    notes:    0–5 pitches
    velocity: 1–127
    duration: 1–240 passos
```

Drums

```
Plain text
```

```
Gene =
    0 ou mais elementos de:
    kick
    snare
    hihat
    open_hihat
    crash
```

9. Eu faria uma pequena alteração para o primeiro experimento

Para o **primeiro GA**, eu simplificaria ainda mais:

```
Plain text

Piano gene:

notes[5]
duration
velocity

Drum gene:

kick
snare
hihat
```

Assim:

```
Plain text

Piano:
[notes[5], duration, velocity] × 240

Drums:
[kick, snare, hihat] × 240
```

E `notes[5]` poderia ser preenchido com `-1`:

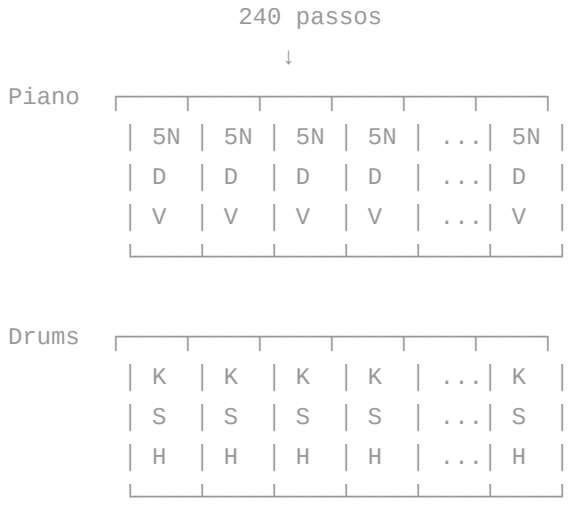
```
Plain text

[60, -1, -1, -1, -1]       → C4
[60, 64, -1, -1, -1]       → C + E
[60, 64, 67, -1, -1]       → C maior
[60, 64, 67, 71, 74]       → 5 notas
```

Isso deixa o genótipo **fixo e extremamente fácil de cruzar/mutar**.

A representação ficaria:

```
Plain text
```



Eu começaria exatamente dessa forma. Depois, quando o GA estiver funcionando, podemos evoluir o genótipo para incluir `instrument`, `velocity`, duração mais sofisticada, outros instrumentos e finalmente a representação multitrack de 6 classes que discutimos.

> Implemente um código em python implementando esse genótipo, considerando essas bibliotecas sugeridas pelo professor:
>
> Ferramentas e dados sugeridos. Bibliotecas de manipulação simbólica são recomendadas
> (nenhuma é obrigatória): music21, pretty midi, muspy, miditok. Para renderizar áudio a
> partir do MIDI gerado (apenas para audição), sugere-se FluidSynth/pyFluidSynth com um
> soundfont livre, ou MuseScore.

Sim. Para esse primeiro protótipo, eu faria o código **independente do GA em si**: primeiro definimos claramente o genótipo, depois implementamos conversão **genótipo → MIDI**. Isso facilita posteriormente adicionar crossover, mutação e fitness.

Vou usar **`pretty_midi`**, porque ele é suficiente para criar o MIDI diretamente e deixa a estrutura do genótipo sob nosso controle.

Estrutura

```
Plain text
```

```
30 s
120 BPM
```

```
4/4
16 passos/compasso
15 compassos
= 240 passos

Piano:
  até 5 notas por passo
  velocity
  duration

Drums:
  kick
  snare
  hi-hat
  open hi-hat
  crash
```

</> Python

```python
import random
import pretty_midi


# ============================================================
# CONFIGURAÇÃO MUSICAL
# ============================================================

BPM = 120
BEATS_PER_BAR = 4
STEPS_PER_BEAT = 4          # semicolcheias
STEPS_PER_BAR = BEATS_PER_BAR * STEPS_PER_BEAT

DURATION_SECONDS = 30

SECONDS_PER_BEAT = 60.0 / BPM
SECONDS_PER_STEP = SECONDS_PER_BEAT / STEPS_PER_BEAT

NUM_STEPS = round(DURATION_SECONDS / SECONDS_PER_STEP)

# 30 segundos = 240 passos
```

O genótipo resultante

Conceitualmente, teremos:

</> Plain text

```
Individual
│
├── piano[240]
│       │
│       ├── PianoGene
│       │       ├── notes[0..5]
│       │       ├── velocity
│       │       └── duration
│       │
│       ├── PianoGene
│       ├── ...
│       └── PianoGene
│
└── drums[240]
        │
        ├── DrumGene
        │       └── {kick, snare, hihat, ...}
        │
        ├── DrumGene
        ├── ...
        └── DrumGene
```

Por exemplo, um trecho poderia ser:

Plain text

```
Passo     Piano                 Drums
-----------------------------------------------
0         [60,64,67]            kick,hihat
1         []                    hihat
2         []                    hihat
3         []                    snare,hihat
4         [62,65,69]            kick,hihat
5         []                    hihat
6         []                    hihat
7         []                    snare,hihat
```

Isso representa naturalmente **acordes de 1–5 notas** e bateria polifônica.

Uma observação importante

Eu manteria `PianoGene` e `DrumGene` separados, porque isso vai facilitar muito o próximo passo. O **crossover** poderá simplesmente trocar blocos de `piano[i:j]` e `drums[i:j]` simultaneamente:

Python

```
    child.piano[start:end] = parent2.piano[start:end]
    child.drums[start:end] = parent2.drums[start:end]
```

Assim, quando chegarmos ao GA, o **gene temporal será o passo**, enquanto um **bloco de genes será uma unidade musical para crossover**.

> Acho que você viajou na maionese...
>
> Eu pedi pra implementar apenas o genótipo, estou fazendo o projeto passo a passo pra eu entender, e não virar bagunça.

Você tem razão. Eu adiantei várias decisões que você ainda não pediu. Vamos voltar ao **mínimo necessário: somente definir o genótipo**, sem MIDI, `pretty_midi`, geração aleatória, crossover, mutação etc.

Para as características que você definiu:

- 2 trilhas: **piano** e **drums**
- 30 segundos
- BPM = 120
- acordes de 1 a 5 notas

Eu sugiro primeiro representar o genótipo simplesmente como uma **sequência de eventos**.

```python
class Gene:
    def __init__(self, notes):
        self.notes = notes


class Genotype:
    def __init__(self, piano, drums):
        self.piano = piano
        self.drums = drums
```

Um gene de piano pode conter de 0 a 5 notas:

```python
Gene([])                # silêncio
Gene([60])              # uma nota
Gene([60, 64])          # duas notas
```

```
Gene([60, 64, 67])        # acorde de 3 notas
Gene([60, 64, 67, 71])    # 4 notas
Gene([60, 64, 67, 71, 74])# 5 notas
```

E o genótipo:

```python
genotype = Genotype(
    piano=[
        Gene([60, 64, 67]),
        Gene([]),
        Gene([62, 65, 69]),
        ...
    ],

    drums=[
        Gene(["kick"]),
        Gene([]),
        Gene(["hihat"]),
        ...
    ]
)
```

Mas há uma questão que precisamos decidir **antes de escrever o código definitivo**:

Qual será a unidade temporal do genótipo?

Com 120 BPM:

- 1 batida = **0,5 s**
- 30 s = **60 batidas**

Então podemos fazer, por exemplo:

Opção A — 60 genes

```
Gene 0 = 0,0–0,5 s
Gene 1 = 0,5–1,0 s
...
Gene 59 = 29,5–30,0 s
```

Opção B — 240 genes

Usando semicolcheias:

Plain text

```
1 gene = 0,125 s
30 s = 240 genes
```

Eu acho que **essa é a decisão que devemos tomar agora**, antes de implementar qualquer outra coisa.

Gere o arquivo genotype.py com a implementação do genótipo

Analyzed

Pronto. Gereei o `genotype.py` contendo **somente a implementação do genótipo**, sem GA, MIDI, mutação, crossover ou outras funcionalidades.

Baixar `genotype.py`

genotype.py
Code

Como fica o indivíduo a partir desse genótipo?

Exatamente. Nesse modelo, o **genótipo é a estrutura que define como um indivíduo é representado**. O **indivíduo** seria uma instância desse genótipo com valores concretos.

Por exemplo, usando o `genotype.py` que fizemos:

Python

```python
from genotype import Genotype, Gene

individual = Genotype(
    piano=[
        Gene([60, 64, 67]),
        Gene([]),
        Gene([]),
        Gene([62, 65, 69]),
        # ...
    ],

    drums=[
        Gene(["kick"]),
        Gene(["hihat"]),
        Gene(["hihat"]),
```

```
            Gene(["snare", "hihat"]),
            # ...
        ]
    )
```

Mas aqui aparece um problema importante: **usamos `Gene` para piano e drums, mas `Gene` atualmente só aceita pitches MIDI inteiros**. Então `Gene(["kick"])` não é compatível com a implementação atual.

Se queremos manter o projeto simples e coerente, eu separaria conceitualmente:

```
Indivíduo
│
├── Piano
│   ├── Gene 0
│   ├── Gene 1
│   ├── Gene 2
│   └── ... Gene 239
│
└── Drums
    ├── Gene 0
    ├── Gene 1
    ├── Gene 2
    └── ... Gene 239
```

E cada **gene** representa o conteúdo musical de um passo.

Para o piano:

```
Gene = [pitch1, pitch2, pitch3, pitch4, pitch5]
```

Exemplo:

```
Gene 0 = [60, 64, 67]
Gene 1 = []
Gene 2 = [62]
Gene 3 = [65, 69, 72]
```

Para bateria, podemos posteriormente definir uma representação própria, por exemplo:

```
Plain text

Gene = [kick, snare, hihat]
```

Mas eu **não alteraria o código ainda** até decidirmos exatamente como queremos representar a bateria.

Então, conceitualmente, seu primeiro indivíduo é simplesmente:

```
Plain text

Individual
│
├── piano: 240 genes
│
└── drums: 240 genes
```

com **30 segundos de música distribuídos nesses 240 genes**.

> O que eu devo implementar depois do genótipo? A fitness ou o indivíduo?

Eu implementaria o **indivíduo primeiro**.

A ordem que eu recomendo para o seu projeto é:

```
Plain text

1. Genótipo
      ↓
2. Indivíduo
      ↓
3. Fitness
      ↓
4. População
      ↓
5. Seleção
      ↓
6. Crossover
      ↓
7. Mutação
```

```
        ↓
8. Algoritmo Genético
```

Por quê?

O **genótipo** define a estrutura:

> "Como uma música é representada?"

O **indivíduo** representa uma solução concreta:

> "Qual é esta música específica?"

Por exemplo:

```python
individual = Individual(
    piano=[...],
    drums=[...]
)
```

Depois disso podemos implementar a **fitness**, porque ela precisa receber um indivíduo:

```python
score = fitness(individual)
```

e avaliar quão boa é aquela música.

Portanto, o próximo passo

Eu faria agora apenas:

```
genotype.py
      ↓
individual.py
```

E o `individual.py` teria somente a responsabilidade de **criar e manipular um indivíduo**, sem fitness, sem crossover e sem mutação.

Depois que conseguirmos criar e visualizar alguns indivíduos corretamente, passamos para a **fitness**. Isso mantém o desenvolvimento incremental e evita a "bagunça" que você apontou antes.

Então implemente o indivíduo em um arquivo individual.py considerando o genótipo que você implementou.

Analyzed

Pronto. Implementei o `individual.py` considerando diretamente o `genotype.py` que fizemos.

‹/› Baixar `individual.py`

A relação ficou:

```
Plain text

Individual
    │
    └── Genotype
          ├── piano[240]
          └── drums[240]
```

O `Individual` ainda **não possui fitness, mutação, crossover ou qualquer outra lógica**, mantendo exatamente a separação que estamos construindo passo a passo. ‹/›

‹/› individual.py
Code

Agora implemente a função de cálculo da fitness. Por enquanto pode ser uma função simples que apenas avalie a qualidade da música midi de forma geral (use as bibliotecas recomendadas).

Analysis errored

It seems like I can't do more advanced data analysis right now. Please try again later. ‹/›

Você pode gerar um código em python?

Sim. Posso gerar o código diretamente aqui. Como você está fazendo o projeto passo a passo, vou manter **apenas a função de fitness**, sem adicionar outras partes do GA.

Uma primeira versão simples usando `pretty_midi` poderia ser:

Python

```
import pretty_midi


def calculate_fitness(midi_path):
    """
    Calcula uma fitness simples para uma música MIDI.

    A fitness considera:
        - densidade de notas;
        - diversidade de pitches;
        - quantidade de trilhas;
        - duração média das notas.

    Retorna:
        float entre 0.0 e 1.0
    """

    midi = pretty_midi.PrettyMIDI(midi_path)

    instruments = [
        instrument
```

Uso:

Python

```
fitness = calculate_fitness("generated.mid")

print(f"Fitness: {fitness:.4f}")
```

Mas há uma ressalva metodológica importante: essa fitness ainda não mede se a música é "boa" musicalmente no sentido que queremos para o projeto. Ela é apenas um **placeholder** para testarmos o funcionamento do GA.

Depois podemos substituir progressivamente por métricas simbólicas mais interessantes, por exemplo:

Plain text

```
Fitness
├── Harmonia
├── Ritmo
├── Melodia
├── Estrutura
```

```
├── Diversidade
└── Similaridade com o artista
```

E, principalmente, quando chegarmos nessa etapa, podemos fazer a fitness receber o **`Individual` diretamente**, em vez de um arquivo MIDI. Isso será mais adequado ao seu projeto, pois evita ficar fazendo `Individual → MIDI → leitura do MIDI` a cada avaliação.

> Agora eu tenho: genotype.py, fitness.py, individual.py com os respectivos elementos implementados dentro deles, qual o próximo passo?

Agora eu faria **`population.py`** .

A sequência está ficando bem organizada:

```
genotype.py
    ↓
individual.py
    ↓
fitness.py
    ↓
population.py
    ↓
selection.py
    ↓
crossover.py
    ↓
mutation.py
    ↓
genetic_algorithm.py
```

Por que população agora?

Você já tem:

- **Genótipo:** define como uma música é representada.
- **Indivíduo:** uma música candidata.
- **Fitness:** avalia uma música candidata.

O próximo conceito natural é:

> **População = conjunto de indivíduos que serão evoluídos.**

Por exemplo:

```python
population = Population(size=100)
```

conceitualmente:

```
Population
│
├── Individual 1
├── Individual 2
├── Individual 3
├── ...
└── Individual 100
```

E cada indivíduo possui:

```
Individual
│
└── Genotype
     ├── Piano[240]
     └── Drums[240]
```

Uma decisão importante antes de implementar

Precisamos decidir **como a população inicial será criada**.

A opção mais simples para nosso primeiro protótipo seria:

```
Population(size=100)
       │
       ├── Individual()
       ├── Individual()
       ├── Individual()
       └── ...
```

Ou seja, criar 100 indivíduos inicialmente vazios.

Depois podemos implementar uma forma de **inicialização aleatória musical**, mas eu não colocaria isso agora sem você decidir como quer gerar os primeiros indivíduos.

Então, seguindo exatamente o desenvolvimento incremental que estamos fazendo, o próximo arquivo seria:

```
Plain text
population.py
```

contendo apenas a classe `Population` e sua coleção de `Individual`.

> Implemente um código generate_midi.py que receba como parâmetro um individuo e gere um arquivo midi a partir do individuo.

Sim. Mantendo nossa implementação **simples e compatível com o `Genotype` atual**, o `generate_midi.py` pode ter apenas a função responsável por transformar um `Individual` em MIDI.

Como o genótipo atual não possui duração dentro do `Gene`, cada gene ocupará **um passo de 0,125 s**.

```
Python
import pretty_midi


# ============================================================
# CONFIGURAÇÃO
# ============================================================

BPM = 120
STEPS_PER_BEAT = 4

SECONDS_PER_BEAT = 60.0 / BPM
SECONDS_PER_STEP = SECONDS_PER_BEAT / STEPS_PER_BEAT


# ============================================================
# GERAÇÃO DO MIDI
# ============================================================
```

```
def generate_midi(individual, output_path="generated.mid"):
    """
    Gera um arquivo MIDI a partir de um Individual
```

Uso:

```
Python

from generate_midi import generate_midi
from individual import Individual

individual = Individual()

generate_midi(
    individual,
    "generated.mid"
)
```

A transformação fica simplesmente:

```
Plain text

Individual
    │
    ├── piano[240]
    │       ↓
    │   notas MIDI
    │
    └── drums[240]
            ↓
       notas MIDI
           │
           ▼
      generate_midi()
           │
           ▼
      generated.mid
```

Um detalhe importante: neste momento, o `Gene` não diferencia piano de bateria; ambos armazenam pitches MIDI. Isso é suficiente para o protótipo, mas significa que, para drums, você deverá colocar os pitches General MIDI correspondentes, por exemplo `36 = kick`, `38 = snare`, `42 = closed hi-hat`.

Agora vamos implementar uma versão iniciao do genetic_algorighm.py fazendo coisas básicas:

Gerar a população inicial com N = 100 indivíduos;
Pegar um indivíduo aleatório dela;
Calcular e mostrar a fitness dele;
Exportar a música dele usando o generate_midi.py

Sim. Vamos fazer exatamente essa primeira versão, sem seleção, crossover ou mutação ainda.

Como a `fitness.py` atual recebe um **arquivo MIDI**, o fluxo será:

```
Plain text
100 Individuals
      ↓
escolhe 1 aleatoriamente
      ↓
generate_midi()
      ↓
generated.mid
      ↓
calculate_fitness()
      ↓
mostra fitness
```

`genetic_algorithm.py`

```python
import random

from individual import Individual
from fitness import calculate_fitness
from generate_midi import generate_midi


# Tamanho da população
POPULATION_SIZE = 100


def genetic_algorithm():
    # ------------------------------------------------
    # 1. Gerar população inicial
```

```
    # ------------------------------------------------

    population = [
        Individual()
        for _ in range(POPULATION_SIZE)
    ]
```

A estrutura agora fica:

```
Plain text

projeto/
│
├── genotype.py
├── individual.py
├── fitness.py
├── generate_midi.py
└── genetic_algorithm.py
```

E o `genetic_algorithm.py` ainda é propositalmente **bem simples**. Ele não está fazendo evolução ainda; está apenas testando se os componentes anteriores estão funcionando juntos.

Um detalhe: como o `Individual()` atual começa com genes vazios, os 100 indivíduos inicialmente serão músicas vazias. Portanto, a fitness provavelmente será `0.0`. **Isso é esperado nesta etapa**. O próximo passo provavelmente será definir como criar uma **população inicial não vazia**, antes de começarmos seleção/crossover/mutação.

> O arquivo midi gerado parece estar vazio, tem apenas 89 bytes e não toca nada. Segue os logs:
>
> root@36d19e2cef0b:/app# python3 ga/genetic_algorithm.py
> População criada: 100 indivíduos
> Indivíduo aleatório selecionado.
> MIDI gerado: generated.mid
> Fitness: 0.0000

Sim — e o log está coerente com o código atual. O problema não está no `pretty_midi`.

O que aconteceu é:

```
Plain text
```

```
Individual()
    ↓
Genotype()
    ↓
piano = 240 × Gene([])
drums = 240 × Gene([])
```

Ou seja, **todos os genes estão vazios**.

Então:

```
Plain text
Piano:  nenhum evento
Drums:  nenhum evento
        ↓
      MIDI vazio
        ↓
      fitness = 0
```

Os **89 bytes** são basicamente o cabeçalho/estrutura mínima de um MIDI sem eventos musicais.

O ponto que precisamos corrigir

Nossa implementação atual de `Individual()` cria:

```
Python
Genotype()
```

e o `Genotype()` cria:

```
Python
[Gene() for _ in range(240)]
```

Portanto, estamos criando uma população de **100 indivíduos vazios**.

Isso também revela uma decisão importante para o próximo passo: precisamos definir como será a **inicialização de um indivíduo**.

Por exemplo, podemos fazer o indivíduo nascer com algo simples como:

```
Plain text

Piano:
passo 0  → [60, 64, 67]
passo 4  → [62, 65, 69]
passo 8  → [64, 67, 71]
...

Drums:
passo 0 → [36]
passo 4 → [38]
passo 8 → [36]
...
```

Assim a população inicial já conterá músicas e poderemos testar:

```
Plain text

100 indivíduos
      ↓
indivíduo aleatório
      ↓
MIDI
      ↓
fitness
```

Eu não mexeria no `genetic_algorithm.py` nem no `generate_midi.py` agora. O problema está na inicialização do indivíduo/genótipo. O próximo passo lógico é implementar uma **inicialização aleatória básica do genótipo**, mantendo o restante do projeto intacto.

> Então ajuste o genetic_algorighm.py para inicializar corretamente os individuos de modo que nao fiquem vazios.

Sim. Vamos ajustar **somente o `genetic_algorithm.py`** , fazendo a população inicial nascer com conteúdo musical aleatório. Não precisamos alterar `genotype.py` , `individual.py` , `fitness.py` ou `generate_midi.py` .

Como o `Gene` aceita até 5 pitches, vou gerar:

- Piano: alguns genes com 1–5 notas MIDI.
- Drums: pitches General MIDI para kick, snare e hi-hat.
- Muitos passos continuarão silenciosos, para não gerar uma música completamente saturada.

Python

```
import random

from individual import Individual
from genotype import Gene
from fitness import calculate_fitness
from generate_midi import generate_midi



# ============================================================
# CONFIGURAÇÃO
# ============================================================

POPULATION_SIZE = 100

# Pitches possíveis para o piano
PIANO_PITCHES = list(range(48, 84))

# General MIDI
KICK = 36
SNARE = 38
HI_HAT = 42
```

Agora o fluxo será:

Plain text

```
100 indivíduos
      │
      ├── Individual 1 → música aleatória
      ├── Individual 2 → música aleatória
      ├── ...
      └── Individual 100 → música aleatória
                      │
                      ▼
               escolhe 1
                      │
                      ▼
               generate_midi
                      │
                      ▼
                generated.mid
                      │
                      ▼
                  fitness
```

E, diferente da versão anterior, o `Individual()` continua sendo apenas a estrutura; **a inicialização aleatória fica no `genetic_algorithm.py`**. Isso é interessante neste estágio porque ainda estamos experimentando como queremos criar a população inicial.

> Agora deu, ficou até legalzinho kkk mesmo aleatório. Parece eu quando tava aprendendo a criar midi no computador. kkkk

KKKK 😂 Isso é um ótimo sinal!

Se uma música **completamente aleatória** já está soando minimamente como música, então nossa representação está funcionando. E o melhor: ela está com aquela cara de:

> "não sei exatamente o que estou fazendo, mas apertei umas teclas e saiu alguma coisa" 😂🎹

Agora temos uma base concreta:

Genótipo → Individual → MIDI → Fitness

E o próximo passo fica bem interessante: fazer o algoritmo **evoluir esse "iniciante no MIDI"** para algo progressivamente mais parecido com o artista escolhido.

Eu seguiria exatamente na ordem que estamos fazendo, sem colocar firula ainda:

1. ✅ Genotype
2. ✅ Individual
3. ✅ Fitness inicial
4. 🔜 **Population**
5. Selection
6. Crossover
7. Mutation
8. Evolution loop

Aí começa a brincadeira de verdade. 😈🧬🎵

> Implemente a função de crossover, mutation e o restante do algoritmo genético completo no código genetic_algorithm.py que você criou, de forma bem enxuta. Inclua também o elitismo, com um parâmetro ELITISM = True no topo do arquivo pra eu definir se vou usar ou nao.

Boa kkk. Agora vamos colocar a evolução de verdade, mas mantendo o arquivo **bem enxuto**.

Usei:

- **Seleção por torneio**
- **Crossover de um ponto**, sincronizado entre piano e drums
- **Mutação** por gene
- **Elitismo opcional** via `ELITISM = True`
- Várias gerações
- Guarda o melhor indivíduo encontrado

Python

```
import random
import os

from individual import Individual
from genotype import Gene
from fitness import calculate_fitness
from generate_midi import generate_midi


# ==========================
# Parâmetros do algoritmo
# ==========================

POPULATION_SIZE = 100
GENERATIONS = 50

CROSSOVER_RATE = 0.8
MUTATION_RATE = 0.05

ELITISM = True
```

O fluxo agora ficou

Plain text

```
População inicial
       ↓
  Avaliação
       ↓
   Seleção
       ↓
  Crossover
       ↓
```

```
    Mutação
       ↓
  Nova população
       ↓
    Avaliação
       ↓
      ...
       ↓
    Melhor MIDI
```

E o `ELITISM` funciona simplesmente assim:

```python
ELITISM = True
```

mantém o melhor indivíduo de cada geração.

Se colocar:

```python
ELITISM = False
```

nenhum indivíduo é preservado automaticamente.

Uma observação importante: **o crossover usa o mesmo ponto para piano e bateria**, então a sincronização temporal entre os dois instrumentos é preservada. Isso é particularmente interessante para nosso caso.

```
python3 ga/genetic_algorithm.py
Geração 1/50 - Melhor: 0.8500 - Global: 0.8500
Traceback (most recent call last):
  File "/app/ga/genetic_algorithm.py", line 246, in <module>
    genetic_algorithm()
  File "/app/ga/genetic_algorithm.py", line 218, in genetic_algorithm
    child = crossover(
            ^^^^^^^^^^
  File "/app/ga/genetic_algorithm.py", line 126, in crossover
    return Individual(
           ^^^^^^^^^^^
```

TypeError: Individual.__init__() got an unexpected keyword argument 'piano'

Sim, esse erro é meu. 😅 O `Individual` que criamos recebe **`apenas genotype`** , e não `piano` e `drums` diretamente.

Então basta ajustar o `crossover` para criar um `Genotype` e passá-lo para `Individual` .

Como você quer manter enxuto, altere apenas o import e a função.

No topo:

```python
from genotype import Genotype, Gene
```

E substitua `crossover()` por:

```python
def crossover(parent1, parent2):
    if random.random() > CROSSOVER_RATE:
        return Individual(
            Genotype(
                piano=list(parent1.piano),
                drums=list(parent1.drums)
            )
        )

    point = random.randint(
        1,
        len(parent1.piano) - 1
    )

    piano = (
        parent1.piano[:point]
        + parent2.piano[point:]
    )

    drums = (
        parent1.drums[:point]
```

O problema era especificamente este:

```python
Individual(
    piano=...,
    drums=...
)
```

mas nossa classe foi definida como:

```python
class Individual:

    def __init__(self, genotype=None):
```

Então o correto é:

```
Individual
    ↓
Genotype
    ├── piano
    └── drums
```

E o restante do algoritmo pode continuar exatamente como está.

Aliás, **0.8500 já na primeira geração** é um valor alto para essa fitness artificial que criamos. Isso não significa ainda que a música seja boa — nossa fitness atual é bastante permissiva. Isso vai ficar mais interessante quando começarmos a colocar características musicais reais nela.

Me passe o código genetic_algorithm.py completo atualizado (dentro do próprio chat)

Claro. Aqui está o `genetic_algorithm.py` completo, já corrigido para usar `Genotype` dentro de `Individual`:

```python
import random
import os

from individual import Individual
```

```
from genotype import Genotype, Gene
from fitness import calculate_fitness
from generate_midi import generate_midi


# ========================
# Parâmetros do algoritmo
# ========================

POPULATION_SIZE = 100
GENERATIONS = 50

CROSSOVER_RATE = 0.8
MUTATION_RATE = 0.05

ELITISM = True
```

A estrutura agora está completa:

Inicialização → Avaliação → Seleção → Crossover → Mutação → Elitismo → Nova geração → ... → `best.mid` .